

INSTITUTO FEDERAL
do Piauí

Campus Floriano

Aula 8: Interfaces

Prof. Ronaldo Borges



Gemini Notebook



A Metáfora do Contrato

Uma **interface** define um **contrato**. Ela não dita como o objeto faz algo, apenas o que ele deve fazer.

Em TS, interfaces são puramente construções **de tempo de compilação** (não geram código JavaScript final). Elas garantem a forma (shape) dos dados.



Duck Typing (Tipagem Pato): Se algo se parece com um pato, nada como um pato e faz quack como um pato, então deve ser um pato! (O TS avalia se o objeto atende aos requisitos estruturais, independente de uma declaração explícita prévia).

Gemini Notebook

Anatomia de uma Interface em TypeScript

Palavra-chave de declaração (substitui 'class' ou 'type').

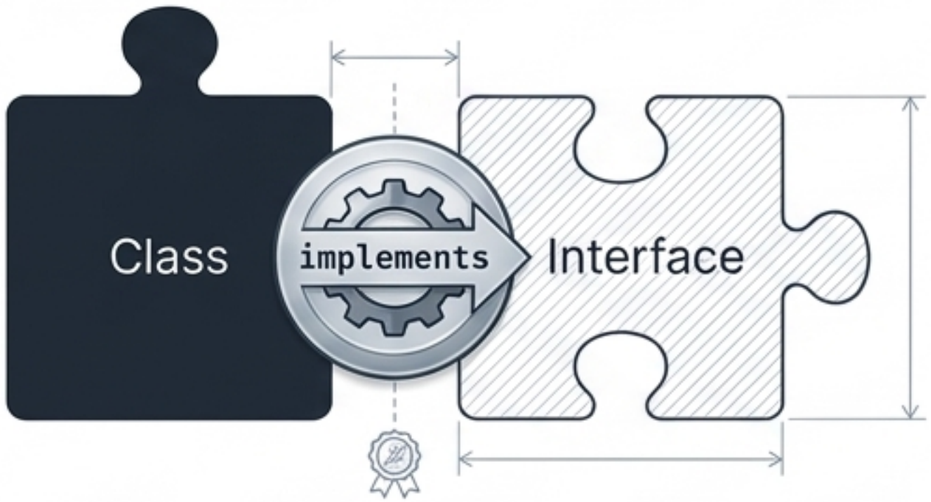
```
interface Person {
  name: string;
  age: number;
  greet(): void;
}
```

Assinatura de atributo (define o tipo, mas não o valor).

Assinatura de método (define parâmetros e retorno, sem corpo {}).

Gemini Notebook

Assinando o Contrato: Implementação



Classes utilizam a palavra-chave `implements` para assinar o contrato. Se a classe falhar em prover todas as propriedades ou métodos exigidos, o compilador TypeScript gerará um erro instantâneo.



CODE SNIPPET

```
class Employee implements Person {
    name: string;
    age: number;

    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    greet() {
        console.log(`Olá, meu nome é ${this.name}`);
    }
}
```

Gemini Notebook

Refinando o Contrato: Flexibilidade Flexibilidade e Segurança



Propriedades Opcionais (?)

Usado para campos que não são estritamente necessários.

CODE SNIPPET

```
interface Friend {
    name: string;
    favoriteColor?: string; // Opcional
}
```



Somente Leitura (readonly)

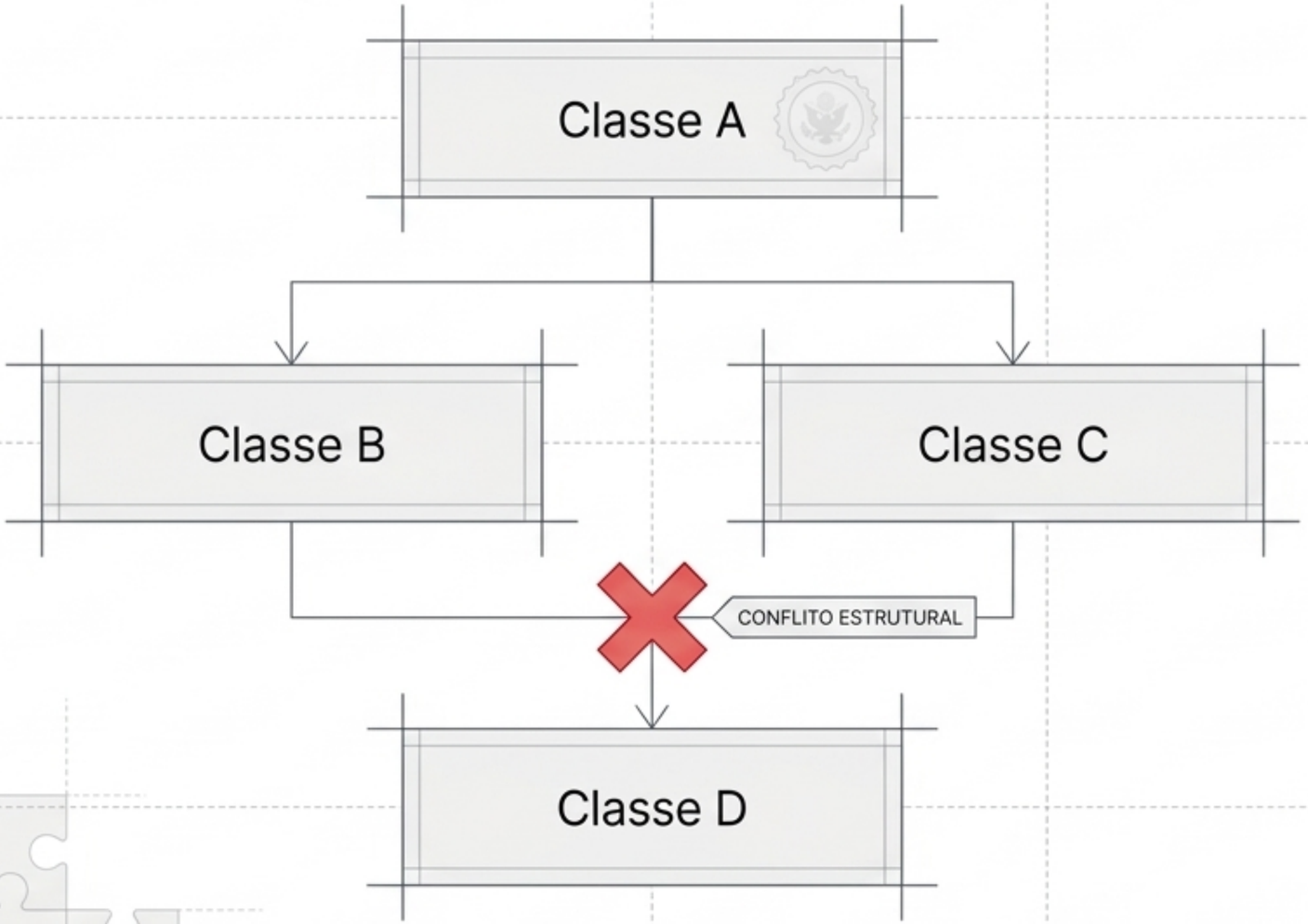
Garante imutabilidade após a criação do objeto.

CODE SNIPPET

```
interface Point {
    readonly x: number;
    readonly y: number;
}
```

Gemini Notebook

O Paradoxo da Herança Múltipla



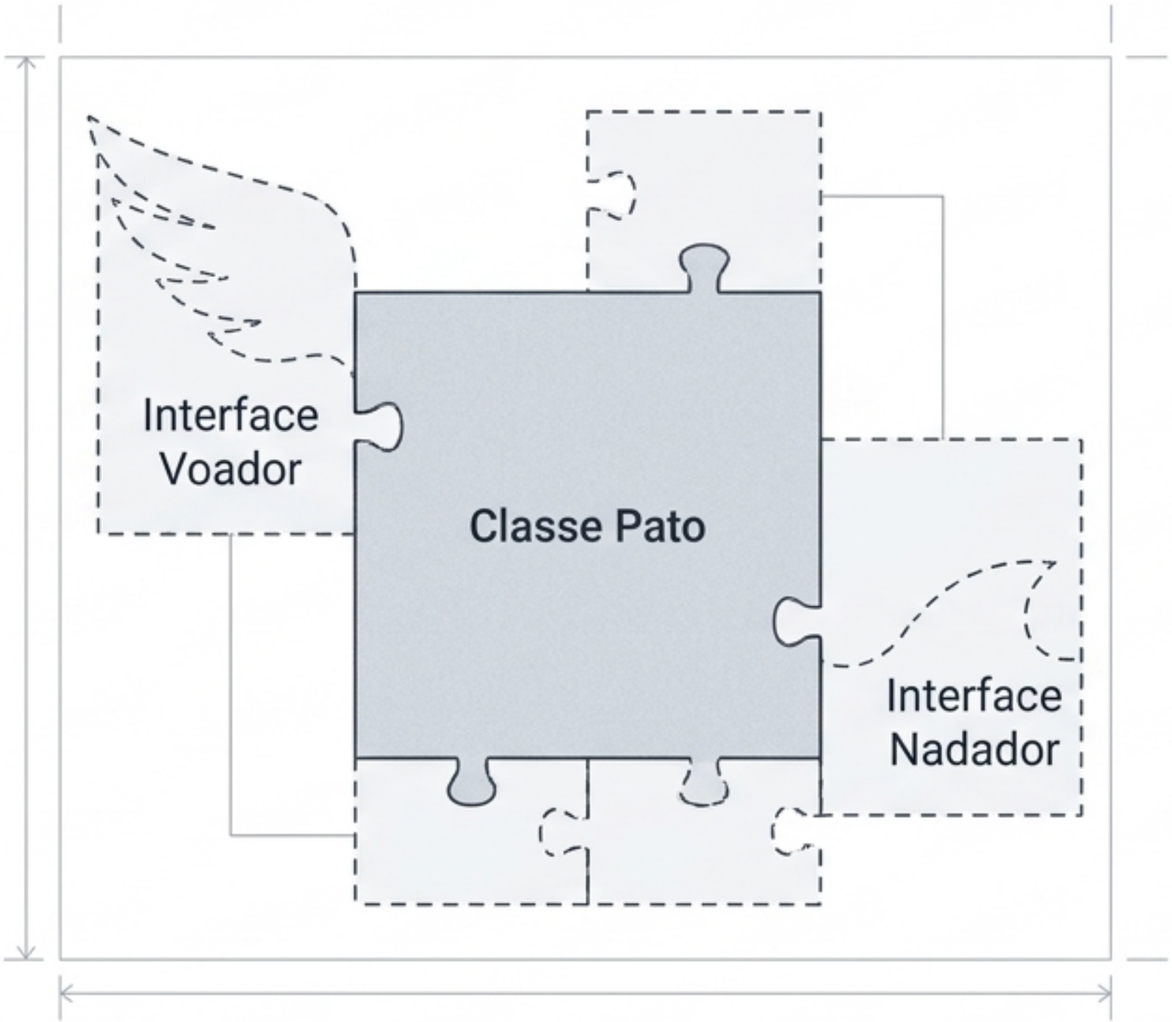
Em linguagens como Java e TypeScript, uma classe não pode herdar de múltiplas classes base simultaneamente simultaneamente (extends A, B gera erro).

O Problema:

Se duas classes base possuírem o mesmo método com implementações diferentes, a classe filha não saberá qual versão executar. Isso causa conflitos de estado e implementação.

Gemini Notebook

A Solução: Múltiplos Contratos



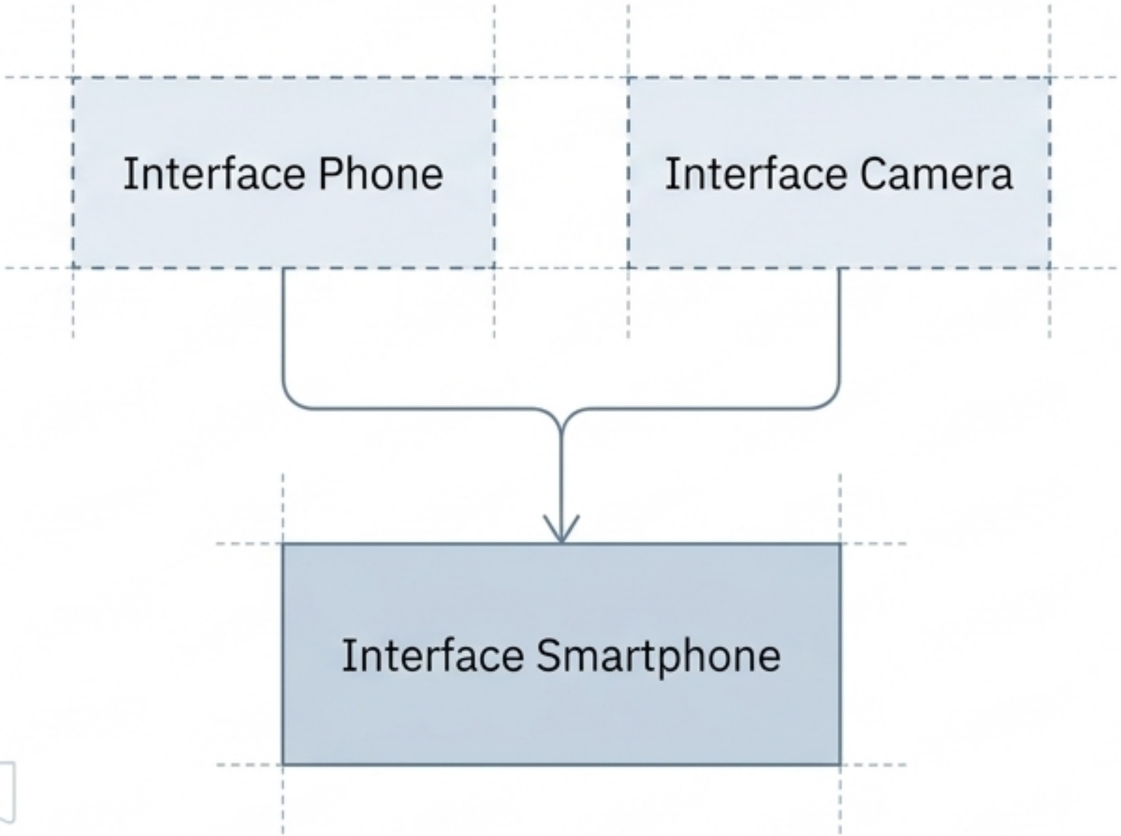
Embora a herança múltipla de classes seja proibida, uma classe pode assinar infinitos contratos (interfaces). Como interfaces não possuem implementação, não há risco de colisão.

```
interface Voador { voar(): string; }
interface Nadador { nadar(): string; }

class Pato implements Voador, Nadador {
  voar() { return "Estou voando!"; }
  nadar() { return "Estou nadando!"; }
}
```

Gemini Notebook

Construindo sobre Contratos (Extends)



- Interfaces podem herdar de outras interfaces usando a palavra-chave **extends**.
- Diferente das classes, uma interface pode estender MÚLTIPLAS interfaces simultaneamente, mesclando suas assinaturas em um único contrato maior.

```
interface Phone { call(): void; }
interface Camera { takePhoto(): void; }

interface Smartphone extends Phone, Camera {
    installApp(): void;
}
```

Gemini Notebook

Matriz de Comparação: Classes Abstratas vs. Interfaces

	Classe Abstrata	Interface
Pode ser instanciada diretamente?	❌ Não	❌ Não
Pode conter implementação real de métodos?	✓ Sim	❌ Não
Permite modificadores de acesso (private, protected)?	✓ Sim	❌ Não
Permite herança múltipla no TypeScript?	❌ Não	✓ Sim
Existe no código JavaScript compilado (runtime)?	✓ Sim	❌ Não (Desaparece)

Gemini Notebook

Combinando Forças no Mundo Real

É possível especializar uma classe (através de herança) e, simultaneamente, assumir **novos contratos** (através de interfaces).

```
class Funcionario {
  constructor(public nome: string) {}
}

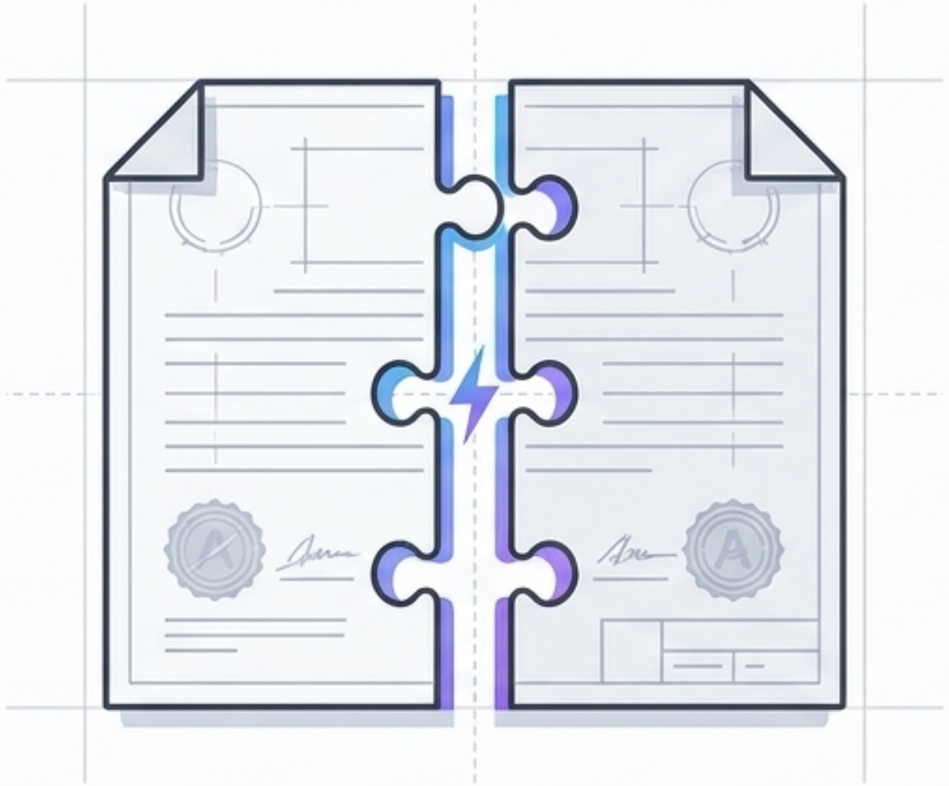
interface AprovadorDeDespesas {
  aprovar(valor: number): boolean;
}

class Gerente extends Funcionario implements AprovadorDeDespesas {
  aprovar(valor: number): boolean {
    return valor < 5000;
  }
}
```

Regra: A palavra-chave `extends` deve sempre vir antes de `implements`.

Gemini Notebook

O Superpoder Exclusivo: Declaration Merging



Diferente de Type Aliases, as Interfaces podem ser reabertas. Se você declarar a mesma interface múltiplas vezes, o TypeScript unirá todas as declarações automaticamente. (Ideal para estender bibliotecas de terceiros).

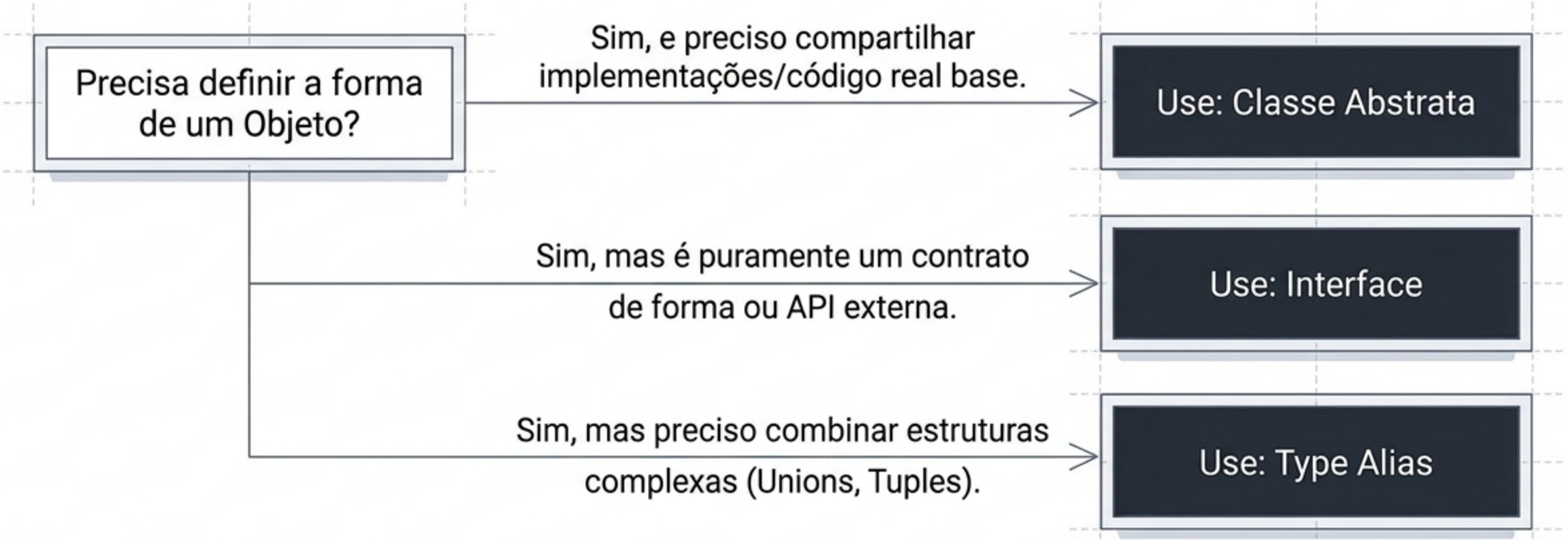
```
interface User { name: string; }

// Reabrindo a interface mais adiante no código:
interface User { age: number; }

// O TypeScript mescla ambas automaticamente:
let user: User = {
    name: "Petya",
    age: 18
};
```

Gemini Notebook

Síntese de Decisão Arquitetural



Regra

Regra de Ouro do TypeScript: Favoreça Interfaces para contratos estruturais, pois elas garantem segurança no desenvolvimento e desaparecem no build, garantindo zero impacto no desempenho final do JavaScript.

Gemini Notebook

Referências Bibliográficas

GRONER, Loiane. Estruturas de dados e algoritmos com JavaScript. 2. ed. São Paulo: Novatec Editora, 2018.

MICROSOFT. TypeScript Language Specification. Disponível na documentação oficial do TypeScript.

ZAKAS, Nicholas C. Princípios de Orientação a Objetos em JavaScript. São Paulo: Novatec Editora, 2022.

Material de aula adaptado das notas do Professor.

Gemini Notebook